Datahub MCP Server Integration with Tavro

1. Objective

The objective of this implementation is to integrate the Dathub MCP (Model Context Protocol) Server with the Tavro application so that Tavro can interact with Datahub metadata.The integration uses the official DataHub MCP Server and Tavro acts as a bridge that exposes DataHub MCP tools through its existing MCP server infrastructure.

2. Prerequisites

Before starting the integration, the following prerequisites are required:

2.1 Datahub Environment

A running Datahub instance with:

- DataHub GMS available and accessible
- Required datasets ingested
- Metadata available in DataHub
- Personal Access Token
- Appropriate user permissions

Refer to the official DataHub Quickstart guide for setting up DataHub locally using Docker:

https://docs.datahub.com/docs/quickstart

Ensure DataHub GMS is running and accessible before proceeding with the integration steps below.

2.2 Generate a DataHub Personal Access Token

The sidecar authenticates to DataHub GMS using a Personal Access Token (PAT):

- Log in to your DataHub instance
- Navigate to Settings → Access Tokens
- Click Generate New Token
- Copy the token — you will use it as DATAHUB_GMS_TOKEN in .env

2.3 Tavro Application

- Tavro source code downloaded from the open-source repository

- Docker and Docker Compose installed on your system
- Git installed for version control
- Basic understanding of Docker container networking

2.4 System Requirements

- Disk Space: Approximately 10 GB or more for Docker images and containers
- RAM: Minimum 8GB (16GB recommended)
- Network: Port availability (8080 for Collibra chip, 9001 for MCP server, 9000 for Tavro API)

3. Integration Approach

The integration uses a sidecar architecture. Instead of embedding DataHub logic directly inside Tavro, the official DataHub MCP Server runs as a separate container called:

datahub-mcp-sidecar

Tavro's MCP server:
risk-mcp-server

uses FastMCP proxy functionality to mount and expose DataHub MCP tools.

the DataHub MCP Server is distributed as a standard Python package on PyPI:
mcp-server-datahub>=0.6.0

This package is installed automatically when the Docker image is built from Dockerfile.datahub-mcp. No manual download or binary placement is needed.

4. Integration Steps

4.1 Clone the Tavro Repository

Clone the Tavro source code from:
git clone https://github.com/TavroOrg/tavro.git
cd tavro

4.2 Create Dockerfile.datahub-mcp

Create a new file at the root:
Dockerfile.datahub-mcp

Code:

```
# syntax=docker/dockerfile:1.6
FROM python:3.12-slim
```

```
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt/lists,sharing=locked \
    apt-get update && apt-get install -y --no-install-recommends curl

RUN --mount=type=cache,target=/root/.cache/pip \
    pip install "mcp-server-datahub>=0.6.0" fastmcp

COPY mcp_server/__init__.py ./mcp_server/__init__.py
COPY mcp_server/datahub_http_server.py
./mcp_server/datahub_http_server.py

EXPOSE 8085

# The official server exposes /health when running in HTTP mode
HEALTHCHECK --interval=15s --timeout=5s --start-period=30s --retries=3
\
    CMD curl -sf http://localhost:8085/health || exit 1

CMD ["python", "-m", "mcp_server.datahub_http_server"]
```

This container installs the official DataHub MCP server package and FastMCP runtime.

4.3 Create DataHub HTTP Server

Create:
mcp_server/datahub_http_server.py

```
import os

from mcp_server_datahub.__main__ import create_app

if __name__ == "__main__":
    port = int(os.getenv("DATAHUB_MCP_PORT", "8085"))
    print(f"Starting official DataHub MCP server (mcp-server-datahub)
on port {port}")
    print(f"DataHub GMS URL: {os.getenv('DATAHUB_GMS_URL', 'not
set')}")
```

```
    mcp = create_app()
    mcp.run(
        transport="http",
        host="0.0.0.0",
        port=port,
        show_banner=False,
        stateless_http=True,
    )
```

5. Docker Compose Integration

Two changes are needed in docker-compose.yml.

5.1 Add the datahub-mcp-sidecar service

Add the following new service at the end of the services block:

```
datahub-mcp-sidecar:
    build:
      context: .
      dockerfile: Dockerfile.datahub-mcp
    container_name: datahub-mcp-sidecar
    restart: unless-stopped
    environment:
      # Internal Docker DNS — reach DataHub GMS via shared network (no host port needed)
      DATAHUB_GMS_URL: ${DATAHUB_GMS_URL:-http://datahub-gms-quickstart:8080}
      DATAHUB_GMS_TOKEN: ${DATAHUB_GMS_TOKEN:-}
      DATAHUB_MCP_PORT: "8085"
      TOOLS_IS_MUTATION_ENABLED:
${DATAHUB_TOOLS_IS_MUTATION_ENABLED:-false}
    ports:
      - "8085:8085"   # optional: expose on host for local debugging
    networks:
      - default          # tavro_default — so risk-mcp-server can reach us
      - datahub_network   # datahub_default — so we can reach datahub-gms
```

Also add the external network declaration at the bottom of docker-compose.yml, alongside the existing volumes block:

```
networks:
  datahub_network:
    external: true
    # Run: docker network ls | grep datahub
```

```
  # and set this to the actual network name (usually "datahub_default")
  name: ${DATAHUB_NETWORK_NAME:-datahub_network}
```

Why two networks?
The datahub-mcp-sidecar acts as a network bridge. It joins both the Tavro Docker network (default) and the external DataHub Docker network (datahub_network). This means:

- risk-mcp-server can reach the sidecar via Docker DNS at http://datahub-mcp-sidecar:8085/mcp
- The sidecar can reach DataHub GMS via Docker DNS at http://datahub-gms-quickstart:8080

No host ports need to be exposed for internal communication.

5.2 Update the risk-mcp-server service

Find the existing risk-mcp-server service and make two additions:

Add to its environment block:

DATAHUB_MCP_URL: http://datahub-mcp-sidecar:8085/mcp

Add to its depends_on block:

```
    datahub-mcp-sidecar:
      condition: service_started
```

6. Environment Configuration

Open env_sample.txt and add the following block:

```
# ----------------------------
# DataHub MCP sidecar
# ----------------------------
# DATAHUB_GMS_URL=http://datahub-gms-quickstart:8080
# DATAHUB_GMS_TOKEN=
# Set true only when the DataHub MCP sidecar should expose metadata mutation tools
# such as add_tags, add_terms, add_owners, set_domains, and update_description.
DATAHUB_TOOLS_IS_MUTATION_ENABLED=false
```

Then copy env_sample.txt to .env and fill in your actual values:

```
DATAHUB_GMS_URL=http://datahub-gms-quickstart:8080
DATAHUB_GMS_TOKEN=<your_datahub_personal_access_token>
DATAHUB_TOOLS_IS_MUTATION_ENABLED=false
DATAHUB_NETWORK_NAME=datahub_default
```

Finding the DataHub network name: Run docker network ls | grep datahub on the host. The network name is typically datahub_default. Set DATAHUB_NETWORK_NAME to match.

7. Update Tavro MCP Server

Tavro uses FastMCP's proxy feature to dynamically discover and mount all DataHub MCP tools at startup.

Open mcp_server/server.py and make the following change.

7.1 Add the DataHub proxy in the lifespan function

Find the existing lifespan function (or add it if it doesn't exist) and add the DataHub proxy mount:

```
@asynccontextmanager
async def lifespan(app: Starlette):
    # Mount all official DataHub MCP tools from the sidecar onto core.
    # FastMCP.as_proxy discovers tools lazily — no hardcoded tool names needed.
    datahub_url = os.getenv("DATAHUB_MCP_URL", "http://datahub-mcp-sidecar:8085/mcp")
    from fastmcp.server import create_proxy
    datahub_proxy = create_proxy(datahub_url, name="DataHub")
    core.mount(datahub_proxy)
    print(f"DataHub MCP proxy mounted from {datahub_url}")

    async with (google_app.lifespan(app), zitadel_app.lifespan(app)):
        Yield
```

7.2 Build and Start the Environment

Run:
docker compose up -d --build

Verify that:

- datahub-mcp-sidecar starts successfully
- risk-mcp-server starts successfully and logs:
  DataHub MCP proxy mounted from http://datahub-mcp-sidecar:8085/mcp
- No container exits unexpectedly

8. Frontend Changes

8.1 Update buildSystemPrompt.ts

The AI assistant's base system prompt needs to be updated so the LLM knows DataHub is in scope and should use MCP tools instead of claiming it lacks access to catalog data.

Open tavro_app/src/services/buildSystemPrompt.ts and update the BASE constant at the top of the file. Add the DataHub-aware instructions shown below:

```
const BASE = `You are Tavro AI Assistant — an intelligent assistant embedded in Tavro, an enterprise AI governance and operations platform.
You are concise, specific, and grounded in the context provided.
Never make up data. If you don't know something, say so and suggest where to find it.
Format responses clearly using bullet points and bold text where helpful.
When MCP tools are available for external enterprise catalogs such as DataHub or Collibra, those sources are in scope. Use the available MCP tools for questions about datasets, schemas, lineage, glossary terms, tags, owners, domains, data products, dashboards, documents, or other metadata assets. Do not claim you lack access to Snowflake, DataHub, or catalog metadata before checking MCP tools.`;
```

Why: Without this, the LLM would respond to questions like "what datasets are in DataHub?" with "I don't have access to DataHub" even when the MCP tools are available and ready to be called.

8.2 Update mcpClient.ts

Two changes are needed in tavro_app/src/services/mcpClient.ts.

8.2.1 Add the _prepareToolArgs method

Add the following method to McpClientService:

```
private async _prepareToolArgs(name: string, args: Record<string, any> = {}):
Promise<Record<string, any>> {
    const schema = await this._getToolSchema(name);
    const properties = schema?.properties || {};
    const required: string[] = schema?.required || [];
    const supportsOriginalPrompt = Boolean(properties.original_prompt ||
required.includes('original_prompt'));
    const toolArgs: Record<string, any> = { ...args };

    if (supportsOriginalPrompt) {
        // Tavro-owned tools use this for audit/context, but proxied third-party
        // MCP tools such as DataHub's search reject unexpected arguments.
        toolArgs.original_prompt = (args.original_prompt && String(args.original_prompt).trim())
            ? args.original_prompt
```

```
            : `User requested ${name} via Dashboard UI`;
    } else {
        delete toolArgs.original_prompt;
    }

    return toolArgs;
}
```

Then update callTool() to use it — replace the line that builds toolArgs:
const toolArgs = await this._prepareToolArgs(name, args);

8.2.2 — Add DataHub routing guidance to the tool prompt

Inside _llmChatWithTools, add the DataHub catalog routing rule to the tool guidance block that is injected into the system prompt:

```
### Data catalog routing
Questions about DataHub, Snowflake datasets, schemas, lineage, glossary terms, tags, owners, domains, data products, dashboards, documents, or ecommerce metadata are in scope when catalog MCP tools are available. Use tools such as `search`, `get_entities`, `list_schema_fields`, `get_lineage`, `get_lineage_paths_between`, `search_documents`, or `grep_documents` instead of saying you lack access.
```

This rule appears inside the ## MCP Tool Usage — STRICT RULES block so the LLM sees it alongside the available tool list at the start of every chat turn.

8.2.3 — Clear tool cache on invalidateCache()

Add this._mcpTools = null; to the invalidateCache() method:

```
invalidateCache(): void {
    this._agentCache = null;
    this._useCaseCache = null;
    this._agentCacheGen++;
    this._useCaseCacheGen++;
    this._agentDetailCache.clear();
    this._riskSummaryCache.clear();
    this._useCaseDetailCache.clear();
    this._mcpTools = null;   // ← add this line
}
```

9. DataHub MCP Tools Exposed in Tavro

Because Tavro uses FastMCP's proxy, all tools provided by the official mcp-server-datahub package are automatically exposed through Tavro's MCP server — no hardcoded list is maintained.

The set of available tools depends on the installed version of mcp-server-datahub (pinned to >=0.6.0) and the DATAHUB_TOOLS_IS_MUTATION_ENABLED setting.

Read-only tools (always available):

- search — Full-text search across DataHub entities
- get_dataset_details — Retrieve schema, lineage, and metadata for a dataset
- get_entity — Fetch any DataHub entity by URN
- list_domains — List configured data domains
- list_tags — List available tags
- list_glossary_terms — List business glossary terms
- get_lineage — Get upstream/downstream lineage for an entity

Mutation tools (only when DATAHUB_TOOLS_IS_MUTATION_ENABLED=true):

add_tags — Add tags to a dataset
add_terms — Add glossary terms to a dataset
add_owners — Assign ownership
set_domains — Assign a dataset to a domain
update_description — Update dataset or field descriptions

10. Tool Purpose

Tool Name	Purpose
search	Full-text search across all DataHub entities (datasets, dashboards, pipelines, etc.)
get_lineage	Retrieve upstream or downstream lineage for any entity (datasets, columns, dashboards, etc.) with filtering, query-within-lineage, pagination, and hop control.
get_dataset_queries	Fetch real SQL queries referencing a dataset or column—manual or system-generated—to understand usage patterns, joins, filters, and aggregation behavior.
get_entities	Fetch detailed metadata for one or more entities by URN; supports batch retrieval for efficient inspection of search results.
list_schema_fields	List schema fields for a dataset with keyword filtering and pagination, useful when search results truncate fields or when exploring large schemas.

get_lineage_paths_between	Retrieve the exact lineage paths between two assets or columns, including intermediate transformations and SQL query information.
add_tags / remove_tags	Add or remove tags from entities or schema fields (columns). Supports bulk operations on multiple entities.
add_terms / remove_terms	Add or remove glossary terms from entities or schema fields. Useful for applying business definitions and data classification.
add_owners / remove_owners	Add or remove ownership assignments from entities. Supports different ownership types (technical owner, data owner, etc.).
set_domains / remove_domains	Assign or remove domain membership for entities. Each entity can belong to one domain.
update_description	Update, append to, or remove descriptions for entities or schema fields. Supports markdown formatting.
add_structured_properties / remove_structured_properties	Manage structured properties (typed metadata fields) on entities. Supports string, number, URN, date, and rich text value types.

11. Authentication

The DataHub MCP Server authenticates to DataHub GMS using a Personal Access Token (PAT):

Before generating a PAT, ensure token-based authentication is enabled in DataHub by setting:

METADATA_SERVICE_AUTH_ENABLED=true

DATAHUB_GMS_TOKEN=<your_datahub_personal_access_token>
To generate a PAT: log in to your DataHub instance → Settings → Access Tokens → Generate New Token.

The token must belong to a user with read access to the required DataHub entities. For mutation tools, the user also needs write permissions on the relevant entities.

The Tavro risk-mcp-server does not authenticate to the sidecar directly — it communicates over the internal Docker network using the service DNS name datahub-mcp-sidecar. The sidecar holds the DataHub credentials.

12. Validation

The integration was validated by checking that:

- The datahub-mcp-sidecar container starts and passes its health check at http://localhost:8085/health
- The risk-mcp-server logs confirm: DataHub MCP proxy mounted from http://datahub-mcp-sidecar:8085/mcp
- A search call returns results from DataHub